

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE
SÃO PAULO**

TIAGO JARRUGE SARAIVA

**SISTEMA DE GERENCIAMENTO DE SERVIÇOS DE HOTE-
LARIA**

CAMPOS DO JORDÃO

ANO

RESUMO

Neste documento encontra-se uma documentação detalhada sobre o processo de desenvolvimento um sistema de gestão de serviços de hotelaria e um guia completo de seu uso. O sistema possui como objetivo conceder aos usuários um ecossistema de ferramentas que proporcionam facilidade, agilidade e organização nas operações internas críticas em serviços de alojamento. As ferramentas atuais cobrem a criação de quartos, realização de check-in e check-out, monitoramento de finanças, configuração de valores por temporada, monitoramento de disponibilidade de produtos terceiros que podem ser adquiridos pelo cliente final, etc. Além disso, o projeto também tem uma camada de administração e monitoramento para envolvidos no desenvolvimento e manutenção do sistema, como gerentes, programadores, administradores, etc, que permite operar de forma ágil e prática com cobranças, inscrição de novos hotéis, inscrição de novos funcionários, etc.

Palavras-Chave: Hotel, PMS, Sistema, Hotelaria, Estadia

ABSTRACT

This document contains detailed documentation on the development process of a hotel services management system and a complete guide to its use. The system aims to provide users with an ecosystem of tools that offer ease, speed, and organization in critical internal operations in accommodation services. Current tools cover room creation, check-in and check-out, financial monitoring, seasonal pricing configuration, monitoring of the availability of third-party products that can be purchased by the end customer, etc. In addition, the project also has an administration and monitoring layer for those involved in the development and maintenance of the system, such as managers, programmers, administrators, etc., allowing for agile and practical operation with billing, registration of new hotels, registration of new employees, etc.

Keywords: Hotel, PMS, System, hospitality, stay

LISTA DE SIGLAS

IFSP	Instituto Federal de Educação, Ciência e Tecnologia de São Paulo
XML	<i>Extensible Markup Language</i>
PMS	<i>Product Management System</i>
API	<i>Application Programming Interface</i>
CORS	<i>Cross-Origin Resource Sharing</i>
DBa- aS	<i>Database as Service</i>
FOHB	<i>Fórum de Operadores Hoteleiros do Brasil</i>
HMAC	<i>Hash-based Message Authentication Code</i>
HTTP	<i>Hypertext Transfer Protocol</i>
IFSP- CJO	IFSP Câmpus Campos do Jordão
PDF	<i>Portable Document Format</i>
PNPM	<i>Performant Node Package Manager</i>
SaaS	<i>Software as a Service</i>
SGBD	<i>Sistema Gerenciador de Banco de Dados</i>
SQL	<i>Structured Query Language</i>
UC	<i>Use Case (caso de uso)</i>
UI	<i>User Interface (interface do usuário)</i>
UML	<i>Unified Modeling Language</i>

SUMÁRIO

1	INTRODUÇÃO _____	7
1.1	Objetivos _____	8
1.2	Justificativa _____	8
1.3	Aspectos Metodológicos _____	9
2	FUNDAMENTAÇÃO TEÓRICA _____	11
2.1	Sistemas de Gestão Hoteleira e PMS _____	11
2.2	Sistemas Web _____	11
2.3	Arquitetura Cliente-Servidor _____	12
2.4	Banco de Dados Relacional e PostgreSQL	13
2.5	Banco de Dados em Nuvem, Supabase e DBaaS	14
2.6	Modelo de Tenancy em Sistemas SaaS	15
2.7	Metodologias Ágeis	15
2.8	UML na Modelagem de Sistemas	16
2.9	Testes de Software	17
2.10	Controle de Versão	18
2.11	Linguagens e Frameworks de Desenvolvimento	18
3	PROJETO PROPOSTO _____	19
3.1	Recursos _____	20
3.1.1	Recursos do sistema	20
3.1.1.1	Hotéis	20
3.1.1.2	Usuários	21
3.1.1.3	Roles	23

3.1.1.4	Permissões	24
3.1.2	Recursos de Clientes	25
3.1.2.1	Quartos	25
3.1.2.2	Clientes	26
3.1.2.3	Reservas e Estadias	27
3.1.2.4	Produtos	29
3.1.2.5	Painel Financeiro	30
3.1.2.6	Temporadas e Tarifas por Temporada	31
3.2	Casos de Uso	32
3.3	Arquitetura e código	58
3.3.1	Estrutura do Projeto	58
3.3.2	Servidor	60
3.3.3	Cliente	60
3.3.4	Site Institucional	61
3.3.5	Banco de Dados	62
4	CONCLUSÃO	63

1 INTRODUÇÃO

No Brasil e na maioria do Mundo, a demanda sobre o setor de hospedagem apresenta fortíssima tendência de crescimento. O Panorama da Hotelaria Brasileira de 2026, desenvolvido pela HotelInvest e pelo Fórum de Operadores Hoteleiros do Brasil (FOHB), aponta um pipeline ativo de R\$ 13,6 bilhões em investimentos hoteleiros até o final de 2026. Esse aporte prevê a abertura de 178 novos hotéis e cerca de 26 mil unidades habitacionais.

Porém, mesmo diante de um mercado tão grande, ainda existem instalações que carecem de sistemas apropriados de gestão, e essas lacunas no setor abrem uma oportunidade imensa de investimento no setor de software para empresas de alojamento.

Embasado nessa demanda, este projeto propõe o desenvolvimento de um sistema para hotéis, pousadas, resorts, entre outros serviços de alojamento, que possa cobrir todas as demandas desses tipos de serviço, desde as mais básicas até demandas específicas, com o intuito de ser um sistema completo para qualquer tipo de cliente.

O sistema é oferecido como SaaS, com planos compartilhados em ambiente multi-tenant e planos dedicados em ambiente single-tenant.

Embora o projeto seja projetado de forma a ser profissionalmente viável de ser utilizado, ainda é um sistema desenvolvido para fins educacionais e pode não ser suficiente para implementação em cenário real, possivelmente precisando de adaptações e novas implantações para viabilização de uso profissional.

O projeto apresentado pode estar incompleto nesta versão de entrega, considerando que a demanda da disciplina de Projeto Integrador 1 envolvia a inicialização da produção da documentação sobre o sistema, cobrindo tudo o que já foi implementado durante as atividades da disciplina CJOPRO1. Dessa forma, alguns recursos que serão implementados apenas no semestre seguinte podem ainda não estar documentados.

1.1 Objetivos

Este trabalho tem por objetivo propor o desenvolvimento de um PMS de hotelaria, com foco em serviços desde pequeno até grande porte.

Para a consecução deste objetivo foram estabelecidos os objetivos específicos:

- Executar pesquisa sobre demandas de serviços de hotelaria para planejamento do sistema e banco de dados.
- Realizar um levantamento de requisitos sobre maiores desafios operacionais de serviços de hospedagem, para planejamento de soluções com base não só em padrões de sistemas do tipo, mas necessidades reais.
- Desenvolver uma base de um planejamento de implementação, desde a base do software até features específicas necessárias.
- Estruturar a base do software de forma escalável, segura e organizada, com padrões de projeto bem definidos, como orientação a permissões para renderização de elementos no cliente.
- Desenvolver interface de operações para desenvolvedores do sistema, para facilitar tarefas como gerenciar pagamentos e planos de uso de hotéis, monitorar finanças e manipulação de registros no db.
- Desenvolver ferramentas para uso pelos clientes diretos do sistema e seus funcionários, como recepcionistas, gerentes, administradores, camareiros, etc, que cubram todas as demandas operacionais que um serviço de hospedagem pode ter.

1.2 Justificativa

A falta de um sistema de gerenciamento para serviços de hospedagem pode comprometer severamente a organização do negócio, interferindo negativamente em gerenciamento de funcionários, clientes, quartos, preços, entre outros elementos sempre presentes em empresas deste setor, podendo prejudicar consideravelmente a ordem e eficiência das tarefas e uso de recursos.

Diante desses problemas enfrentados, surge a necessidade de um método de organizar a gestão desses tipos de serviço.

Mediante a isso, o sistema proposto proporcionará aos clientes uma forma de centralizar todas as atividades administrativas, organizacionais e financeiras de seus estabelecimentos em uma única ferramenta digital.

1.3 Aspectos Metodológicos

Neste tópico, são apresentados os principais aspectos metodológicos adotados no desenvolvimento do PMS voltado a sistemas de hotelaria.

As técnicas empregadas no desenvolvimento desta solução foram adquiridas, em sua maioria, no decorrer da formação no curso de Análise e Desenvolvimento de Sistemas do IFSPCJO, por meio dos conteúdos abordados nas disciplinas, de estudos complementares realizados em cursos digitais e materiais disponíveis na internet, além de experiências obtidas em projetos anteriores.

Para o desenvolvimento do projeto, foram adotados princípios da metodologia ágil, caracterizada pelo desenvolvimento iterativo e incremental, com possibilidade de ajustes contínuos ao longo do processo. Essa escolha se justifica pela flexibilidade do método, permitindo que as funcionalidades do sistema fossem planejadas, implementadas, avaliadas e aprimoradas de acordo com as necessidades identificadas durante o desenvolvimento.

O processo de desenvolvimento foi organizado em etapas, envolvendo o levantamento das funcionalidades principais, a modelagem da solução, a definição da arquitetura, a implementação dos módulos do sistema e a realização de testes manuais para validação dos principais fluxos de uso. Entre esses fluxos, destacam-se as operações relacionadas à autenticação, gerenciamento de dados e comunicação entre a aplicação cliente, o servidor backend e o banco de dados.

Para auxiliar na documentação e no planejamento do sistema, foram empregadas técnicas da UML (Unified Modeling Language), com o objetivo de representar de forma padronizada aspectos relacionados à arquitetura, aos atores envolvidos e ao funcionamento geral da solução.

2 FUNDAMENTAÇÃO TEÓRICA

Nesta seção, serão apresentados os principais conceitos necessários para a compreensão do projeto, com o objetivo de estabelecer o embasamento teórico e conceitual relacionado ao tema, à relevância da solução proposta e às decisões adotadas ao longo do desenvolvimento.

2.1 Sistemas de Gestão Hoteleira e PMS

A gestão hoteleira envolve o controle e a organização de diferentes processos que fazem parte do funcionamento de meios de hospedagem, como hotéis, pousadas e outros tipos de estabelecimentos de estadia. Entre essas atividades, encontram-se o gerenciamento de reservas, o controle de acomodações, o cadastro de hóspedes, a comunicação com clientes e a administração de dados operacionais.

Com o aumento da demanda por eficiência no atendimento, o uso de sistemas digitais tornou-se um recurso chave no setor de hotelaria. Esses sistemas permitem reduzir falhas consequentes de controles manuais por meio de automatizações, facilitar processos repetitivos, como check-in, check-out, monitoramento de consumo dos clientes, monitoramento financeiro centralizado, entre outros. Dessa forma, esses produtos digitais contribuem para uma experiência mais eficiente e eficaz tanto para os funcionários quanto para os hóspedes.

Nesse contexto, os sistemas do tipo “PMS” representam uma solução para modernizar e otimizar os processos de empresas do setor.

O PMS se trata justamente de uma classificação de sistema responsável por centralizar e gerenciar as principais operações de uma propriedade hoteleira, reunindo funcionalidades voltadas ao controle de reservas, acomodações, hóspedes, estadias, consumo, disponibilidade e demais processos essenciais para a administração do estabelecimento.

2.2 Sistemas Web

Sistemas web são aplicações desenvolvidas para serem acessadas por meio de navegadores, utilizando a internet ou uma rede local como meio de comunicação. Diferentemente de sistemas instalados diretamente na unidade de memória de um dispositivo específico, as aplicações web permitem que os usuários acessem suas funcionalidades a partir de diferentes máquinas, desde que possuam um navegador, conexão e permissão de acesso.

Esse modelo de sistema é amplamente utilizado por possibilitar maior flexibilidade e facilidade de manutenção. Como a aplicação é disponibilizada em ambiente web, atualizações no sistema são entregues à todos os usuários simultaneamente, visto que, uma vez que o Sistema Web é atualizado, todos os novos acessos a ele serão à versão já atualizada, removendo a necessidade de instalação/atualização manual em cada computador utilizado pelos usuários, o que traz uma facilidade muito grande na entrega de versões melhoradas do sistema sem necessidade de operações individuais em diferentes dispositivos.

No contexto de sistemas de gestão, como um PMS, a abordagem web é especialmente relevante por facilitar a distribuição de versões melhoradas do sistema e correções de problemas, principalmente por sistemas desse tipo normalmente se encontrarem em cenários muito críticos, em que lidam com uma quantidade enorme de dados e sustentam operações essenciais para a integridade do estabelecimento a todo momento, de forma que a dificuldade na manutenção poderia trazer prejuízo às empresas clientes e riscos reais aos usuários.

2.3 Arquitetura Cliente-Servidor

A arquitetura cliente-servidor é um modelo amplamente utilizado no cenário do desenvolvimento de sistemas web, no qual as responsabilidades da aplicação são divididas entre duas partes principais: o cliente e o servidor. O cliente corresponde à interface utilizada pelo usuário para interagir com o sistema, enquanto o servidor é responsável por processar requisições do cliente, aplicar regras de negócio, realizar validações e acessar os dados necessários para o funcionamento da aplicação.

Nesse modelo, o cliente envia solicitações ao servidor por meio de uma rede,

geralmente utilizando protocolos de comunicação como o HTTP. O servidor, por sua vez, recebe essas solicitações, executa os procedimentos necessários mediante a solicitação e retorna uma resposta ao cliente.

Um dos principais benefícios desta arquitetura é a distinção de funções entre o cliente e o servidor, em que o cliente é reponsável por lidar apenas com a interatividade do usuário, exibição de dados, validação básica de entradas e requests para o servidor, enquanto a lógica arbitrária do negócio e acesso ao banco de dados fica no servidor, isolado do cliente de forma a encapsular esse lado crítico do sistema, ajudando a evitar ataques e usos mal intencionados, visto que o cliente só realiza operações que o servidor permite, e o servidor em si não pode ser diretamente manipulado.

Outro benefício importantíssimo desse modelo é a centralização de acesso de dados, pois neste modelo, independente da máquina utilizada para o acesso, con- tanto que o usuário tenha permissão, ele pode acessar um único servidor centraliza- do para adquirir e enviar dados que garantem o funcionamento do sistema.

Em sistemas de gestão, como um PMS, essa arquitetura é essencial por permitir que diferentes usuários acessem dados e funcionalidades do sistema de forma centralizada. Dessa forma, operações como cadastro de hóspedes, controle de reservas e consulta de acomodações podem ser realizadas por meio da comunicação entre a aplicação cliente, o servidor backend e o banco de dados.

2.4 Banco de dados relacional e PostgreSQL

Bancos de dados relacionais são sistemas utilizados para armazenar, organizar e gerenciar informações de forma estruturada. Nesse modelo, os dados são distribuídos em tabelas, compostas por linhas e colunas, nas quais cada linha representa um registro e cada coluna representa um atributo relacionado àquele conjunto de dados.

Uma das principais características dos bancos de dados relacionais é a possibilidade de estabelecer relações entre diferentes tabelas por meio de chaves primárias e chaves estrangeiras. As chaves primárias identificam de forma única cada registro, enquanto as chaves estrangeiras permitem associar dados entre tabelas

distintas. Esse mecanismo favorece a integridade das informações, contribuindo para uma estrutura mais organizada e consistente.

No contexto de um sistema de gestão hoteleira, o uso de um banco de dados relacional é adequado devido à necessidade de armazenar informações interligadas, como dados de hóspedes, reservas, acomodações, usuários, estadias, pagamentos e demais registros operacionais. Como esses dados possuem relações entre si, o modelo relacional permite representar essas associações de maneira clara e controlada.

O PostgreSQL é um Sistema Gerenciador de Banco de Dados Relacional, também conhecido como SGBD, que utiliza a linguagem SQL para manipulação e consulta de dados. Ele é amplamente utilizado em aplicações web por oferecer recursos voltados à consistência, segurança, integridade e confiabilidade no armazenamento das informações.

Neste projeto, o PostgreSQL foi utilizado como base para a persistência dos dados do PMS, permitindo que as informações essenciais do sistema fossem armazenadas de forma estruturada e relacional.

2.5 Banco de dados em nuvem, Supabase e DBaaS

O uso de bancos de dados em nuvem permite que as informações de um sistema sejam armazenadas em servidores remotos, acessíveis por meio da internet ou de redes autorizadas. Esse modelo reduz a necessidade de manter uma infraestrutura local própria para hospedagem do banco de dados.

Nesse contexto, destaca-se o modelo DBaaS, ou Database as a Service, no qual o banco de dados é disponibilizado como um serviço gerenciado por uma plataforma especializada. Esse tipo de solução permite que tarefas relacionadas à infraestrutura, disponibilidade e gerenciamento do ambiente sejam simplificadas, possibilitando que o desenvolvimento do sistema se concentre nas funcionalidades da aplicação.

O Supabase é uma plataforma que oferece recursos de backend como serviço, tendo como uma de suas principais funcionalidades a disponibilização de bancos de dados PostgreSQL em ambiente de nuvem. Dessa forma, ele permite que aplicações utilizem um banco relacional gerenciado sem a necessidade de configurar manualmente toda a infraestrutura de hospedagem.

No contexto deste projeto, o Supabase foi utilizado para hospedar o banco de dados PostgreSQL responsável pela persistência das informações do PMS.

2.6 Modelo de Tenancy em Sistemas SaaS

Em sistemas disponibilizados como serviço, também conhecidos como SaaS, o conceito de tenancy está relacionado à forma como a aplicação organiza e gerencia os dados de diferentes clientes. Nesse contexto, cada cliente, empresa ou organização que utiliza o sistema pode ser chamado de tenant. No caso de um PMS, um tenant pode corresponder a um hotel, pousada, rede hoteleira ou outro estabelecimento contratante da solução.

O modelo single-tenant ocorre quando cada cliente possui uma instância isolada da aplicação, do banco de dados ou de parte da infraestrutura. Nesse formato, os dados e configurações de um estabelecimento ficam separados dos demais clientes, oferecendo maior isolamento, segurança e possibilidade de personalização. Por outro lado, esse modelo tende a exigir mais recursos de infraestrutura e maior esforço de manutenção, já que cada cliente pode possuir um ambiente próprio.

Já o modelo multi-tenant ocorre quando múltiplos clientes utilizam uma mesma instância da aplicação ou do banco de dados, com separação lógica das informações. Nesse caso, os dados de diferentes estabelecimentos podem coexistir em uma mesma estrutura, desde que existam mecanismos adequados de controle de acesso, identificação do tenant e isolamento das informações. Esse modelo costuma ser mais econômico e escalável, pois permite atender vários clientes utilizando uma infraestrutura compartilhada.

2.7 Metodologias ágeis

As metodologias ágeis correspondem a abordagens de desenvolvimento de software baseadas em ciclos iterativos e incrementais. Diferentemente de modelos mais rígidos e sequenciais, essas metodologias valorizam a adaptação contínua, a entrega gradual de funcionalidades e a possibilidade de ajustes ao longo do processo de desenvolvimento.

Nesse modelo, o sistema pode ser construído em partes menores, permitindo que funcionalidades sejam planejadas, implementadas, testadas e aprimoradas progressivamente. Essa característica favorece a identificação antecipada de problemas, a correção de falhas e a adaptação do projeto conforme novas necessidades são percebidas durante sua evolução.

No contexto de um sistema de gestão hoteleira, como um PMS, a adoção de princípios ágeis é relevante devido à diversidade de funcionalidades envolvidas, como reservas, cadastro de hóspedes, controle de acomodações, autenticação, permissões e comunicação com o banco de dados. Como esses módulos possuem regras e fluxos próprios, o desenvolvimento incremental contribui para organizar melhor a implementação e validar cada parte do sistema de forma gradual.

Dessa forma, as metodologias ágeis auxiliam no desenvolvimento de soluções mais flexíveis e adaptáveis, permitindo que o projeto acompanhe mudanças de requisitos, melhorias identificadas durante os testes e ajustes necessários para atender melhor aos objetivos do sistema.

2.8 UML na modelagem de sistemas

A UML (*Unified Modeling Language*), ou Linguagem de Modelagem Unificada, é uma linguagem visual utilizada para representar diferentes aspectos de um sistema de software. Seu objetivo é auxiliar na documentação, planejamento e comunicação da estrutura e do comportamento de uma aplicação, por meio de diagramas padronizados.

A utilização da UML permite representar elementos importantes do sistema de forma mais clara, facilitando a compreensão das funcionalidades, dos atores envolvidos, dos fluxos de uso e das relações entre os componentes da solução. Dessa

forma, a modelagem contribui para organizar as ideias do projeto antes da implementação e também serve como apoio para a documentação técnica.

Entre os diagramas mais utilizados estão o diagrama de casos de uso (abreviado para UC, use case), que representa as interações entre os usuários e o sistema; o diagrama de classes, que descreve entidades, atributos, métodos e relacionamentos; e o diagrama de atividades, que demonstra o fluxo de execução de determinados processos. Esses recursos ajudam a visualizar o funcionamento do sistema e a identificar requisitos, dependências e possíveis melhorias.

No contexto de um PMS, a UML pode ser utilizada para representar funcionalidades como cadastro de hóspedes, gerenciamento de reservas, controle de acomodações, autenticação de usuários e demais processos operacionais. Assim, a modelagem auxilia na compreensão da solução proposta e contribui para uma implementação mais organizada e alinhada aos objetivos do projeto.

2.9 Testes de software

Os testes de software são práticas utilizadas para verificar se uma aplicação funciona conforme o esperado e atende aos requisitos definidos para o projeto. Eles auxiliam na identificação de erros, comportamentos inesperados e falhas de integração entre os diferentes componentes do sistema, contribuindo para maior confiabilidade, estabilidade e qualidade da solução desenvolvida.

Entre os tipos de testes mais comuns estão os testes manuais, os testes unitários, os testes de integração e os testes funcionais. Os testes manuais consistem na verificação direta das funcionalidades pelo desenvolvedor ou avaliador, simulando ações realizadas por usuários no sistema. Já os testes unitários têm como objetivo validar partes menores e isoladas do código, como funções, métodos ou regras específicas da aplicação.

Os testes de integração, por sua vez, são utilizados para verificar se diferentes partes do sistema funcionam corretamente em conjunto, como a comunicação entre o servidor backend, as rotas da API e o banco de dados. Esse tipo de teste é especialmente relevante em sistemas que dependem da troca constante de informações entre camadas distintas da aplicação.

Também existem os testes funcionais e de interface, que avaliam o comportamento do sistema sob a perspectiva do usuário final. Esses testes verificam fluxos completos de uso, como autenticação, cadastro, consulta, atualização e exclusão de informações. No contexto de um PMS, essa validação é importante porque o sistema lida com operações essenciais, como reservas, hóspedes, acomodações e demais dados operacionais do estabelecimento.

Dessa forma, os testes de software contribuem para reduzir falhas, aumentar a segurança das alterações realizadas e validar se as funcionalidades implementadas estão de acordo com os objetivos do projeto.

2.10 Controle de versão

O controle de versão é uma prática utilizada no desenvolvimento de software para registrar, organizar e acompanhar as alterações realizadas no código-fonte de um projeto. Por meio dessa prática, é possível manter um histórico das modificações feitas ao longo do tempo, identificar quando determinada alteração foi realizada e, se necessário, recuperar versões anteriores do sistema.

Essa abordagem contribui para maior segurança e organização durante o desenvolvimento, pois permite que novas funcionalidades, correções e ajustes sejam realizados de forma progressiva e rastreável. Além disso, o controle de versão reduz o risco de perda de progresso, facilita a comparação entre diferentes estados do código e auxilia na identificação de alterações que possam ter causado falhas ou comportamentos inesperados.

Uma das ferramentas mais utilizadas para controle de versão é o Git, que permite gerenciar o histórico de alterações de maneira distribuída. Em conjunto com plataformas de hospedagem remota, como o GitHub, o Git possibilita armazenar o repositório em ambiente online, facilitando o acesso ao projeto a partir de diferentes máquinas e favorecendo práticas de colaboração, backup e organização do desenvolvimento.

2.11 Linguagens e frameworks de desenvolvimento

No desenvolvimento de aplicações web modernas, a escolha da linguagem e dos frameworks utilizados influencia diretamente a organização, a produtividade e a manutenção do projeto. Neste contexto, o TypeScript se destaca como uma linguagem baseada em JavaScript que adiciona tipagem estática ao desenvolvimento, permitindo maior previsibilidade no código, melhor identificação de erros durante a implementação e maior segurança na manipulação de dados.

Entre os frameworks utilizados no desenvolvimento do projeto, destaca-se o React, uma biblioteca voltada à construção de interfaces de usuário por meio de componentes reutilizáveis. Essa abordagem favorece a organização visual da aplicação e facilita a manutenção de telas e elementos da interface. O Next.js, por sua vez, é um framework baseado em React que amplia suas funcionalidades, oferecendo recursos voltados ao desenvolvimento de aplicações web mais estruturadas, como organização de rotas, renderização de páginas e melhor suporte à criação de sistemas completos.

Para a construção do servidor backend, foi utilizado o Fastify, um framework para aplicações em ambiente Node.js voltado à criação de APIs e serviços web. Ele permite definir rotas, processar requisições, aplicar validações e intermediar a comunicação entre a aplicação cliente e a camada de persistência de dados.

Dessa forma, o uso de TypeScript, React, Next.js e Fastify contribui para a construção de uma aplicação web organizada em diferentes camadas, separando a interface do usuário, a lógica de comunicação e os serviços responsáveis pelo processamento das informações do sistema.

3 PROJETO PROPOSTO

Nesta sessão, será dada uma descrição minuciosa sobre o projeto proposto por esse documento, tratando em detalhes todos os recursos do sistema implementados até agora e planejamentos de implementações futuras, além de justificativa para todas as decisões arquiteturais e escolha de ferramentas.

Além disso, também serão apresentados nesse capítulo todos os casos de uso e resultados adquiridos até agora, como prints e simulação de fluxos de execuções.

3.1 Recursos

Neste tópico, será tratado sobre todos os recursos disponíveis no sistema, começando pelos recursos de funcionamento do sistema, aqueles que não se relacionam diretamente com as operações dentro de hotéis, mas sim operações de gestão do produto como um todo, e depois, os recursos disponíveis para os hotéis

3.1.1 Recursos do sistema

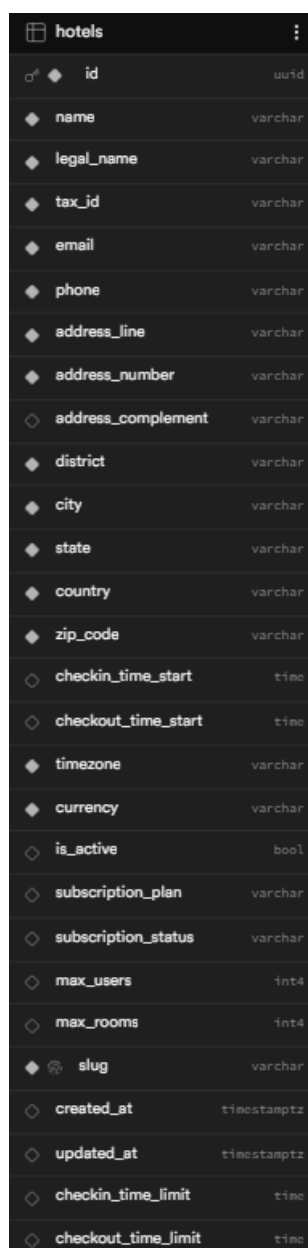
Os recursos de desenvolvimento se tratam sobre recursos manipulados pelos desenvolvedores do projeto.

Esta sessão se tratará sobre quais são os recursos de funcionamento do sistema existentes e como, por onde e quando um desenvolvedor interage com eles, para realizar as operações obrigatórias pra manutenção, cobrança, fiscalização, cadastros, entre outras operações, do produto e dos consumidores.

3.1.1.1 Hotéis

Os Hotéis, ou Tenants, cadastrados nos planos de utilização do sistema são o recurso mais fundamental do sistema, afinal eles são a motivação central do desenvolvimento da solução.

Cada Hotel, dentro do sistema, é representado por um registro na tabela hotels dentro do banco de dados.



hotels		
id	uuid	
name	varchar	
legal_name	varchar	
tax_id	varchar	
email	varchar	
phone	varchar	
address_line	varchar	
address_number	varchar	
address_complement	varchar	
district	varchar	
city	varchar	
state	varchar	
country	varchar	
zip_code	varchar	
checkin_time_start	time	
checkout_time_start	time	
timezone	varchar	
currency	varchar	
is_active	bool	
subscription_plan	varchar	
subscription_status	varchar	
max_users	int4	
max_rooms	int4	
slug	varchar	
created_at	timestampz	
updated_at	timestampz	
checkin_time_limit	time	
checkout_time_limit	time	

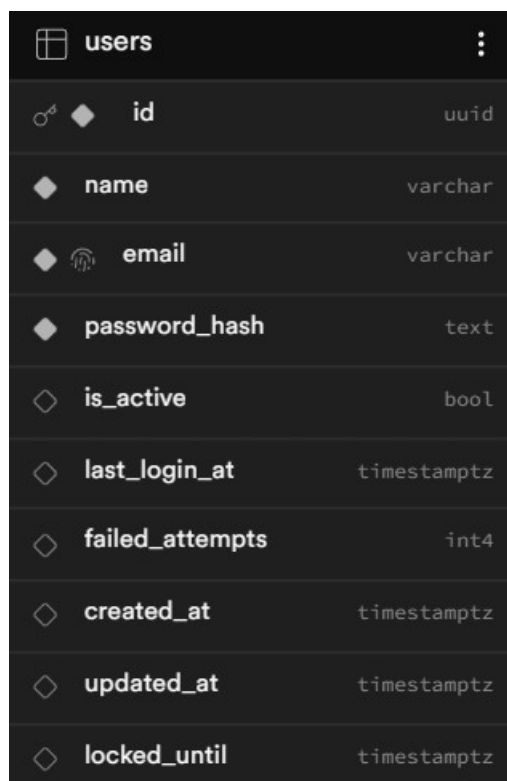
Figura 1 – Tabela hotels (extraída do Supabase)

Cada hotel diferente possui várias outras tabelas relacionadas a ele, como Clientes, Quartos, Reservas, entre outros.

Da perspectiva do sistema, há atualmente 3 casos de uso que envolvem diretamente os hotéis: Cadastro (UC. 1), Remoção do sistema (UC. 2), Edição de dados (UC. 3).

3.1.1.2 Usuários

Os usuários correspondem à todas as pessoas que possuem cadastro no sistema, sejam funcionários diretos da instituição do PMS ou funcionários de um ou mais hotéis cadastrados.

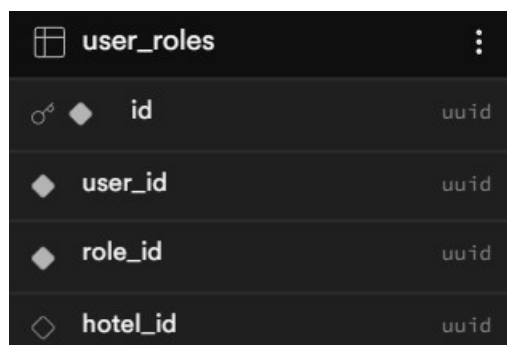


users		
id	uuid	
name	varchar	
email	varchar	
password_hash	text	
is_active	bool	
last_login_at	timestampz	
failed_attempts	int4	
created_at	timestampz	
updated_at	timestampz	
locked_until	timestampz	

Figura 2 – Tabela users (extraída do Supabase)

Todos os usuários do sistema também poder ser associados a um ou mais hotéis, e em cada uma dessas associações, pode possuir um papel diferente a tabela que associa esses três elementos é user_roles.

Exceto colaboradores do PMS, todos os outros usuários vão obrigatoriamente ser relacionados com, no mínimo, uma role em um hotel.



user_roles		
id	uuid	
user_id	uuid	
role_id	uuid	
hotel_id	uuid	

Figura 3 – Tabela user_roles (extraída do Supabase)

Um detalhe importante de ser adicionado sobre o gerenciamento de usuários é que no sistema, atualmente, ele é possível de ser realizado apenas por funcionários com roles de sistema, ou seja, funcionários de hotel só podem ser cadastrados por funcionários do sistema, o que é um comportamento inadequado, visto que a staff de cada hotel deve poder ter autoridade pra registrar seus próprios funcionários, e essa questão será resolvida em breve.

Atualmente, os casos de uso que envolvem diretamente os usuários são Cadastro de usuário (UC. 4), Deletar usuário (UC. 5), Editar dados do usuário (UC. 6), Adicionar roles ao usuário (UC. 7).

3.1.1.3 Roles

As roles representam os papéis atribuíveis aos usuários dentro do sistema, definindo o conjunto de permissões associadas a cada função. Dessa forma, ao receber uma determinada role, o usuário passa a ter acesso às operações e recursos permitidos por ela.

Cada role é formada por uma ou mais permissões, permissões essas que são usadas pelo sistema para validar operações e filtrar elementos que são exibidos pela UI.

Existem 2 classificações gerais de roles, que são as roles do sistema e as roles de hotel. As roles do sistema são constituídas por permissões voltadas ao gerenciamento geral da plataforma e são atribuídas aos colaboradores responsáveis pela administração do PMS. Já as roles de hotel são compostas de permissões que precisam ser vinculadas a um hotel específico, garantindo que os usuários associados a essas funções precisem ser relacionados também a um hotel (de acordo com **Figura 3 – Tabela user_roles**), e então, possam gerenciar apenas os dados pertencentes ao respectivo estabelecimento.

roles	
id	uuid
name	varchar
hotel_id	uuid
role_type	varchar

Figura 4 – Tabela roles (extraída do Supabase)

Também é possível observar que a tabela roles possui o campo hotel_id, que é opcional e tem como objetivo permitir a criação de roles personalizadas para hotéis específicos. As roles com hotel_id nulo representam permissões gerais de hotel, podendo ser associadas a funcionários de qualquer estabelecimento, desde que sejam classificadas como hotel_role (role de hotel), e não como system_role (role de sistema).

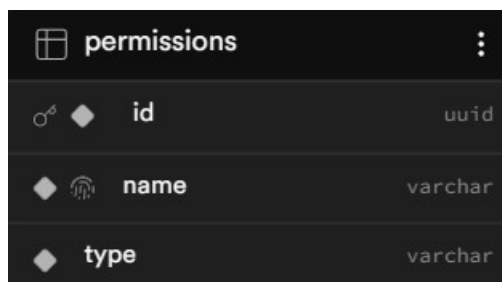
Atualmente, os casos de uso que envolvem diretamente roles são Criação de role (UC. 8), Deletar role (UC. 9), Gerenciar permissões da role (UC. 10), Adicionar roles a um usuário (UC. 7).

3.1.1.4 Permissões

As permissões são entidades que se responsabilizam por declarar para o sistema o que o usuário pode ver dentro da interface gráfica e quais operações pode realizar dentro do PMS.

No código do projeto, todos os recursos que existem podem possuir um atributo que indica qual ou quais permissões um usuário precisa ter para acessar aquele elemento, e em cada operação, o código valida se o usuário conectado tem permissão pra realizar qualquer ação.

No banco de dados, permissões são representadas por uma tabela que registra o nome e o tipo de cada permissão. O tipo se refere a se a permissão é uma permissão relevante à hotel ou a gestão do sistema, e serve pra evitar atribuição errônea de uma permissão se sistema a uma role de hotel.



permissions		
id	uuid	
name	varchar	
type	varchar	

Figura 5 – Tabela roles (extraída do Supabase)

Atualmente, os casos de uso que envolvem diretamente permissões são Criação de permissão (UC. 11), Deletar permissão (UC. 12), Gerenciar permissões de uma role (UC. 10).

3.1.2 Recursos de clientes

Os recursos de clientes são todos os recursos que podem ser usados pelos colaboradores dos hotéis cadastrados no PMS, são as ferramentas da gestão a sistemas de hotelaria propriamente dita.

3.1.2.1 Quartos

Os quartos de um serviço hoteleiro registrados no sistema. Todos os quartos possuem o atributo `hotel_id`, que indica a qual hotel aquele quarto pertence, de forma que apenas roles relacionadas àquele hotel possam gerenciar aos dados do quarto.

rooms		
id	uuid	
hotel_id	uuid	
room_number	text	
room_type	text	
max_occupancy	int4	
base_daily_rate	numeric	
status	room_status	
notes	text	
created_at	timestamptz	
updated_at	timestamptz	

Figura 6 – Tabela rooms (extraída do Supabase)

Os casos de uso disponíveis até agora para quartos são Criar quarto (UC. 13), Deletar quarto (UC. 14), Editar dados do quarto (UC. 15).

3.1.2.2 Clientes

Os clientes diretos dos hotéis, ou seja, as pessoas que se hospedam nos serviços de hospedagem cadastrados no sistema, são armazenados como registro na tabela customers.

A tabela customers armazena os clientes vinculados a um hotel específico. Cada cliente pertence a um único hotel por meio do campo hotel_id, fazendo com que os cadastros sejam isolados entre hotéis. Dessa forma, apenas o campo id é globalmente único no sistema, enquanto informações como e-mail, telefone e documento devem ser únicas apenas dentro do escopo do mesmo hotel.

customers		
id	uuid	
full_name	text	
document_number	text	
document_type	text	
email	text	
mobile_phone	text	
phone	text	
birth_date	date	
nationality	text	
notes	text	
created_at	timestampz	
updated_at	timestampz	
hotel_id	uuid	

Figura 7 – Tabela customers (extraída do Supabase)

Os casos de uso disponíveis até agora para costumers são Criar costumer (UC. 16), Editar dados de costumer (UC. 17), Criar reserva para costumer (UC. 18).

3.1.2.3 Reservas e Estadias

No PMS, as Reservas são registradas no banco de dados na tabela reservati-
ons, e juntamente com a tabela reservations, a tabela stays também faz parte de
forma essencial do fluxo de reservas no sistema.

Além disso, também existem as tabelas stay_consumption, que associam o
consumo de itens durante uma estadia com a respectiva estadia e a tabela stay_-
customers, que associam quais clientes estão ligados a quais estadias.

stay_customers	stay_consumption	stays	reservations
id (pk)	id (pk)	id (pk)	id (pk)
stay_id	product_id	reservation_id	hotel_id
customer_id	quantity	room_id	booking_customer_id
created_at	charged_unit_price	applied_daily_rate	reservation_code
updated_at	item_total_amount	total_price_estimated	guest_count
	consumption_date	created_at	reservation_source
	notes	updated_at	estimated_total_price
	created_at	checkin_date_expected	final_total_price
	updated_at	checkout_date_expected	notes
	stay_id	checkin_date_actual	created_at
		checkout_date_actual	updated_at
		total_paid	total_paid
		stay_status	

Figura 8 – Tabelas do fluxo de reserva e estadia (extraído do Supabase)

O sistema foi arquitetado de forma que a tabela `reservations` representa a reserva principal, ou seja, o registro geral da contratação da hospedagem. Ela guarda dados como o hotel da reserva (`hotel_id`), o cliente responsável pela reserva (`booking_customer_id`), o código da reserva (`reservation_code`), a quantidade de hóspedes, a origem da reserva, os valores totais estimados/finais, observações e pagamentos. Ela funciona como o “cabeçalho” do processo.

A tabela `stays` representa a estadia em si, vinculada a uma reserva e a um quarto específico. Cada estadia possui um `reservation_id` e um `room_id`, além de informações como diária aplicada, valor estimado, datas previstas de check-in/check-out, datas reais de entrada e saída, valor pago e status da estadia. Isso permite que uma única reserva tenha vários quartos, cada um com sua própria estadia.

A tabela `stay_customers` associa os clientes que efetivamente participam de uma estadia. Isso é importante porque o cliente que fez a reserva, registrado em `booking_customer_id`, não necessariamente é o único hóspede. Por exemplo, uma pessoa pode fazer a reserva para si e para outras pessoas, ou até reservar em nome de terceiros. Assim, `stay_customers` funciona como uma tabela intermediária entre estadia e clientes hospedados.

Já a tabela `stay_consumption` registra os consumos realizados durante a

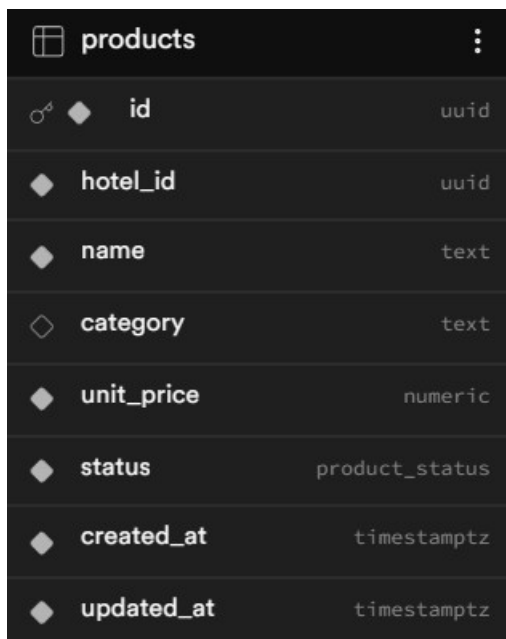
estadia, como produtos, itens de frigobar, serviços ou outras cobranças vinculadas ao quarto. Cada consumo aponta para uma estadia (`stay_id`) e para um produto (`product_id`), armazenando quantidade, preço unitário cobrado, valor total do item, data do consumo e observações. Esses registros podem ser somados ao valor da hospedagem no fechamento da conta.

A staff do hotel possui poder para configurar diversas ferramentas do hotel a respeito de estadia, como intervalo de horário válido para check-in e check-out, método de pagamento (no check-in, no check-out ou na reserva), método de cobrança de consumos extra durante estadia, entre outros, para o sistema se enquadrar às regras da empresa.

Os casos de uso disponíveis até agora para reservas e estadias são Criar reserva para costumer (UC. 18), Realizar check-in de estadia (UC. 19), Realizar check-out de estadia (UC. 20), Registrar consumo em estadia (UC. 21).

3.1.2.4 Produtos

Os produtos disponíveis por um hotel para consumo de clientes também são registrados no sistema e podem ser monitorados.



products		
id	uuid	
hotel_id	uuid	
name	text	
category	text	
unit_price	numeric	
status	product_status	
created_at	timestampz	
updated_at	timestampz	

Figura 9 – Tabela dos produtos (extraído do Supabase)

Os casos de uso disponíveis até agora para produtos de um hotel cadastrado são Registrar consumo em estadia (UC. 20), Registrar produto (UC. 21), Apagar produto (UC. 22), Editar dados de um produto (UC. 23).

3.1.2.5 Painel financeiro

A solução proposta por este projeto também providencia um painel financeiro aos usuários, para acesso do setor financeiro e administrativo de determinado hotel cadastrado.

Todas as operações financeiras realizadas no hotel, como recebimento de pagamento de reservas, pagamento de contas, pagamento de salários, entre outros, são exibidas no painel financeiro, além de outras métricas relevantes, como saldo, pagamentos em aberto, expirados, etc.

financial_transactions		
id		uuid
hotel_id		uuid
type	transaction_type	
category		text
amount		numeric
currency		text
description		text
status	transaction_status	
created_at		timestampz
stay_id		uuid
reservation_id		uuid
payment_method		text
paid_at		timestampz
created_by		uuid
updated_at		timestampz
due_date		date
counterparty		text
cost_center		text
reference_code		text

Figura 10 – Tabela das transações financeiras (extraído do Supabase)

Os casos de uso disponíveis até agora para produtos de um hotel cadastrado são Registrar pagamento (UC. 24), Gerar relatório financeiro (UC. 25).

3.1.2.6 Temporadas e Tarifas pro temporada.

O sistema do PMS permite que os clientes configurem temporadas nas quais desejam aplicar mudanças de tarifa, e depois associar cada temporada com uma categoria de quarto e uma variação no valor do quarto, seja negativa ou positiva.

Assim, é permitido aos colaboradores de um hotel de que eles criem variação nos preços com base na época do ano, aumentando os preços, por exemplo, em época de alta temporada, e abaixando em época de baixa temporada.

season_room_rates	:	seasons	:
id	uuid	id	uuid
season_id	uuid	hotel_id	uuid
hotel_id	uuid	name	text
room_type	text	start_date	date
daily_rate	numeric	end_date	date
created_at	timestampz	is_active	bool
updated_at	timestampz	created_at	timestampz
		updated_at	timestampz

Figura 11 – Tabelas do fluxo de variação de tarifa (extraído do Supabase)

Os casos de uso disponíveis até agora para criação de variação de tarifas para quartos de um hotel cadastrado são Criar temporada (UC. 26), Editar temporada existente(UC. 27), Criar tarifa sob temporada (UC. 28), Associar tarifa de certa temporada com uma categoria de quarto (UC. 29) e Deletar temporada existente (UC. 30)

3.2 Casos de uso

Nesta sessão, serão declarados todos os casos de uso previamente referenciados durante a sessão 3.1

tabela 1 – UC. 1 – Cadastro do hotel

Nome do caso de uso	UC. 1 → Cadastro de hotel
Ator principal	Colaborador do PMS com permissão pra cadastrar Hotel, como gerente e outros agentes que trabalham no processo de venda do serviço
Resumo	Ator principal recolhe os dados do hotel que deseja criar cadastro e insere na interface visual do PMS.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão para cadastro de hotel. Além disso, deve possuir os dados do hotel que irá cadastrar.
Pós-condições	Um novo hotel está cadastrado no sistema.
Fluxo principal	
Ações do Autor	Ações do sistema
1. Autor recolhe os dados do hotel que será cadastrado e insere nos campos apropriados.	
2. Autor confirma envio dos dados	
	3. Servidor recebe os dados do cadastro realizado e valida os dados. Se faltarem campos ou dados inválidos estiverem sido enviados na request, o cadastro falha, desviando para fluxo alternativo 1
	4. Servidor confirma a criação do cadastro e envia feedback positivo para a interface.
Fluxo alternativo 1 – Erro de cadastro	

Ações do Autor	Ações do sistema
	1. Servidor retorna mensagem de erro para o cliente, informando o caráter do erro.

tabela 2 – UC. 2 e UC. 3– Remoção do hotel ou edição de dados do hotel no sistema

Nome do caso de uso	UC. 2 e UC. 3 → Remoção e edição do hotel no sistema
Ator principal	Colaborador do PMS com permissão pra remover e/ou editar dados do Hotel, como gerente e outros agentes que trabalham no processo de venda do serviço
Resumo	Ator principal deseja remover o hotel do sistema diante de cancelamento de cadastro ou outros motivos, ou editar dados mediante a mudanças dos dados do hotel.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão para remoção ou edição do hotel, além disso, deve possuir consentimento do hotel para realizar as operações.
Pós-condições	As alterações no sistema são aplicadas, seja a remoção do hotel ou edição de seus dados.
Fluxo principal	
Ações do Autor	Ações do sistema
1. Autor seleciona um hotel na interface de monitoramento de hotéis do sistema	
2. Autor realiza a operação que deseja, seja deleção do hotel ou mudança de dados.	
	3. Servidor recebe a requisição de remoção do hotel no sistema ou edição dos dados do hotel. No caso de edição de dados, se faltarem campos ou dados inválidos estiverem sido enviados na request de, a edição de dados falha, desviando para fluxo alternativo 1

	4. Servidor confirma execução da operação desejada
Fluxo alternativo 1 – Erro de requisição	
Ações do Autor	Ações do sistema
	1. Servidor retorna mensagem de erro para o cliente, informando o caráter do erro.

tabela 3 – UC. 4 – Cadastro de usuário

Nome do caso de uso	UC. 4 → Cadastro de usuário
Ator principal	Colaborador do PMS com permissão de cadastrar usuário, como gerente e outros agentes que trabalham no processo de gestão do serviço.
Resumo	Ator principal deseja cadastrar um novo usuário do serviço, como por exemplo, um novo colaborador do sistema no PMS, ou um novo funcionário para algum hotel cadastrado.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão para criar usuários, além disso, deve possuir acesso às credenciais do usuário.
Pós-condições	O usuário é criado no sistema com seus dados respectivos.
Fluxo principal	
Ações do Autor	Ações do sistema
1. Autor preenche os campos de cadastro de usuário.	
2. Autor confirma envio da request de criação de cadastro de usuário para servidor	
	3. Servidor recebe a requisição de criação de novo usuário no sistema. Se faltarem campos obrigatórios ou dados

	inválidos estiverem sido enviados na request, o cadastro falha, desviando para fluxo alternativo 1
	4. Servidor confirma execução de criação de novo usuário.
Fluxo alternativo 1 – Erro de requisição	
Ações do Autor	Ações do sistema
	1. Servidor retorna mensagem de erro para o cliente, informando o caráter do erro.

tabela 4 – UC. 5 e UC. 6 – Editar ou deletar usuário

Nome do caso de uso	UC. 5 e UC. 6 → Editar ou deletar
Ator principal	Colaborador do PMS com permissão de editar ou deletar usuário, como gerente e outros agentes que trabalham no processo de gestão do serviço.
Resumo	Ator principal deseja editar ou deletar um usuário do serviço.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão para deletar e/ou editar usuários, além disso, deve possuir consentimento do usuário pra executar a ação.
Pós-condições	Os dados do usuário no sistema são editados, no caso de edição, ou o registro do usuário é apagado, no caso de remoção.
Fluxo principal	
Ações do Autor	Ações do sistema
1. Autor realiza as alterações nos campos do usuário ou seleciona o input de remoção do usuário do sistema.	
2. Autor confirma envio da request de edição ou remoção do usuário para servidor	
	3. Servidor recebe a requisição de edi-

	ção ou remoção de um usuário no sistema. No caso de edição, se faltarem campos obrigatórios ou dados inválidos estiverem sido enviados na request, a edição falha, desviando para fluxo alternativo 1
	4. Servidor confirma execução de criação de novo usuário.
Fluxo alternativo 1 – Erro de requisição	
Ações do Autor	Ações do sistema
	1. Servidor retorna mensagem de erro para o cliente, informando o caráter do erro.

tabela 5 – UC. 7 – Adicionar roles ao usuário

Nome do caso de uso	UC. 7 → Adicionar roles ao usuário
Ator principal	Colaborador do PMS com permissão de adicionar roles ao usuário, como gerente e outros agentes que trabalham no processo de gestão do serviço.
Resumo	Ator principal deseja adicionar roles a usuário do serviço.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão para adicionar roles a usuários, além disso, deve possuir consentimento da instituição a qual aquele usuário pertence para executar a ação.
Pós-condições	O usuário recebe roles novas.
Fluxo principal	
Ações do Autor	Ações do sistema
1. Autor acessa o menu de adição de roles a usuários, seleciona um usuário e as suas novas roles.	
2. Autor confirma envio da request de	

atualização de roles.	
	3. Servidor recebe a requisição de edição de adição de novas roles ao usuário.
	4. Servidor confirma execução de edição de roles ao usuário.

Tabela 6 – UC. 8 – Criação de role

Nome do caso de uso	UC. 8 → Criação de role
Ator principal	Colaborador do PMS com permissão de criar roles, como gerente e outros agentes que trabalham no processo de gestão do serviço.
Resumo	Ator principal deseja criar uma nova role para o sistema ou hotel.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão para criar roles.
Pós-condições	Uma nova role é criada no sistema para poder ser atribuída a usuários.
Fluxo principal	
Ações do Autor	Ações do sistema
1. Autor cria a nova role no menu de criação de roles do dashboard do PMS, selecionando o tipo de role e quais permissões são associadas a ela.	
2. Autor confirma envio da request de criação de role para servidor	
	3. Servidor recebe a requisição de criação de nova role.
	4. Servidor confirma execução de criação de nova role.

tabela 7 – UC. 9 – Deletar role

Nome do caso de uso	UC. 9 → Deletar role
Ator principal	Colaborador do PMS com permissão pra cadastrar Hotel, como gerente e outros agentes que trabalham no processo de venda do serviço
Resumo	Ator principal deleta role existente no sistema.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão para deletar role. Além disso, não pode desejar deletar a uma role que ele mesmo possui, pois essa ação já é bloqueada direto da interface.
Pós-condições	A role é deletada do sistema.
Fluxo principal	
Ações do Autor	Ações do sistema
1. Autor seleciona para deletar determinada role	
	3. Servidor recebe a request de delete da role.
	4. Servidor confirma a remoção da role selecionada e envia feedback de ação concluída para a interface.

tabela 8 – UC. 10 – Gerenciar permissões da role

Nome do caso de uso	UC. 10 → Gerenciar permissões da role
Ator principal	Colaborador do PMS com permissão para editar dados de roles, como gerente e outros agentes que trabalham no processo de venda do serviço.
Resumo	Ator principal seleciona uma role e edita as permissões que ela possui.
Pré-condições	Ator principal deve estar conectado com suas credenci-

	as no sistema e possuir permissão para editar permissões da role. Além disso, não pode possuir as roles às quais deseja editar, pois dentro das regras do PMS, não é possível editar as próprias roles.
Pós-condições	A role é registrada novamente no banco de dados com as alterações às suas permissões salvas.
Fluxo principal	
Ações do Autor	Ações do sistema
1. Autor escolhe uma role para editar: remove e adiciona novas permissões para ela.	
2. Autor confirma a edição da role.	
	3. Servidor recebe a requisição de edição da role.
	4. Servidor confirma a edição das permissões da role e envia feedback positivo para a interface.

tabela 9 – UC. 11 – Criação de permissão

Nome do caso de uso	UC. 11 → Criação de permissão
Ator principal	Colaborador do PMS com permissão de criar permissão, como gerente e outros agentes que trabalham no processo de venda do serviço
Resumo	Ator principal cria nova permissão, seja de sistema ou hotel.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão para criar permissão. Além disso, não pode haver nas suas roles as permissões que deseja editar, pois as regras do PMS impedem isso.
Pós-condições	Uma nova permissão é criada no sistema

Fluxo principal	
Ações do Autor	Ações do sistema
1. Autor insere os dados no menu de criar uma nova permissão para criá-la	
2. Autor confirma envio dos dados	
	3. Servidor recebe os dados do cadastro realizado e valida os dados.
	4. Servidor confirma a criação da nova permissão e envia feedback positivo para a interface.

tabela 10 – UC. 12 – Deletar permissão

Nome do caso de uso	UC. 12 → Deletar permissão
Ator principal	Colaborador do PMS com permissão de deletar permissão, como gerente e outros agentes que trabalham no processo de venda do serviço
Resumo	Ator principal deleta alguma permissão existente, seja de sistema ou hotel.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão para deletar permissão. Além disso, não pode haver nas suas roles as permissões que deseja deletar, pois as regras do PMS impedem isso.
Pós-condições	A permissão deletada é removida do sistema.
Fluxo principal	
Ações do Autor	Ações do sistema
1. Autor escolhe a permissão que deseja deletar e confirma	

	3. Servidor recebe a request de delete de determinada permissão.
	4. Servidor confirma a remoção permissão e envia feedback positivo para a interface.

tabela 11 – UC. 13 e UC. 15 – Criação e edição de quarto.

Nome do caso de uso	UC. 13 e UC.15 → Criação e edição de quarto
Ator principal	Funcionário de um hotel com permissão de criação e/ou edição de quartos.
Resumo	Ator principal recolhe os dados do quarto que deseja criar cadastro e insere na interface visual do PMS.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão para criação e/ou edição de quartos. Além disso, deve possuir os dados do quarto que irá cadastrar.
Pós-condições	Um novo quarto está cadastrado no hotel ao qual o Ator trabalha, ou algum quarto existente é editado.
Fluxo principal	
Ações do Autor	Ações do sistema
1. Autor recolhe os dados do quarto que será cadastrado e insere nos campos apropriados, ou seleciona um quarto existente no seu hotel e modifica os campos desejados.	
2. Autor confirma envio dos dados	
	3. Servidor recebe os dados do cadastro realizado e valida os dados. Se faltarem campos ou dados inválidos estiverem sido enviados na request, o cadastro ou edição falha, desviando para fluxo alternativo 1
	4. Servidor confirma a criação ou edição

	do quarto e envia feedback positivo para a interface.
Fluxo alternativo 1 – Erro de cadastro	
Ações do Autor	Ações do sistema
	1. Servidor retorna mensagem de erro para o cliente, informando o caráter do erro.

tabela 12 – UC. 14 – Deletar quarto.

Nome do caso de uso	UC. 14 → Deletar quarto
Ator principal	Funcionário de um hotel com permissão de deletar quartos.
Resumo	Ator principal seleciona o quarto que deseja deletar e confirma operação.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão para deletar quartos.
Pós-condições	O Quarto deletado é removido do banco de dados e, conseqüentemente, do hotel ao qual o usuário que fez a operação representa.
Fluxo principal	
Ações do Autor	Ações do sistema
1. Ator seleciona um quarto existente e interage com a opção de deleta-lo.	
	3. Servidor recebe a requisição de remoção de determinado quarto do hotel do Ator.
	4. Servidor confirma a remoção do quarto e envia feedback positivo para a interface.

tabela 13 – UC. 16 e UC. 17 – Criação e edição de costumer.

Nome do caso de uso	UC. 13 e UC.15 → Criação e edição de quarto
Ator principal	Funcionário de um hotel com permissão de criação e/ou edição de cliente.
Resumo	Ator principal recolhe os dados do cliente que deseja criar ou editar cadastro e insere na interface visual do PMS.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão para criação e/ou edição de cliente. Além disso, deve possuir os dados e consentimento do cliente que irá cadastrar.
Pós-condições	Um novo cliente está cadastrado no hotel ao qual o Ator trabalha, ou algum cliente existente é editado.
Fluxo principal	
Ações do Autor	Ações do sistema
1. Ator recolhe os dados do cliente que será cadastrado e insere nos campos apropriados, ou seleciona um cliente existente no seu hotel e modifica os campos de seus dados desejados.	
2. Ator confirma envio dos dados	
	3. Servidor recebe os dados do cadastro ou edição realizada e valida os dados. Se faltarem campos ou dados inválidos estiverem sido enviados na request, o cadastro ou edição falha, desviando para fluxo alternativo 1
	4. Servidor confirma a criação ou edição do cliente e envia feedback positivo para a interface.

Fluxo alternativo 1 – Erro de cadastro	
Ações do Autor	Ações do sistema
	1. Servidor retorna mensagem de erro para o cliente do PMS, informando o caráter do erro.

tabela 14 – UC. 18 – Criar reserva para costumer

Nome do caso de uso	UC. 18 → Criar reserva para costumer
Ator principal	Funcionário de um hotel com permissão de criar reserva para cliente, como um recepcionista.
Resumo	Ator principal recolhe os dados da reserva, como clientes envolvidos e quartos desejados para a reserva e insere na interface visual do PMS.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão de criar reserva. Além disso, deve possuir os dados do quarto e clientes que precisará para realizar a reserva, além do consentimento dos clientes.
Pós-condições	Uma nova reserva está criada para determinado quarto em determinado dia para determinado cliente ou grupo de clientes.

Fluxo principal

Ações do Autor	Ações do sistema
1. Ator entra em contato com o cliente que deseja realizar a reserva e recolhe dados, como quarto(s) desejado(s), datas de reserva, quantidade de pessoas na(s) estadia(s), etc.	
2. Ator verifica se clientes já são cadastrados no sistema, se sim, usa os dados já cadastrados, se não, cadastra durante a criação da reserva	
3. Ator confirma no sistema a criação da reserva.	
	3. Servidor recebe os dados da reserva

	realizada e valida os dados. Se faltarem campos ou dados inválidos estiverem sido enviados na request, o cadastro ou-edição falha, desviando para fluxo alternativo 1
	4. Servidor confirma a criação da reserva e envia feedback positivo para a interface.
Fluxo alternativo 1 – Erro de cadastro	
Ações do Autor	Ações do sistema
	1. Servidor retorna mensagem de erro para o cliente, informando o caráter do erro.

tabela 15 – UC. 19 – Realizar check-in de estadia

Nome do caso de uso	UC. 19 → Realizar check-in de estadia
Ator principal	Funcionário de um hotel com permissão de realizar check-in, como um recepcionista.
Resumo	Ator principal realiza check-in em uma estadia reservada previamente por um cliente.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão de realizar check-in. Além disso, o cliente deve pedir para fazer check-in durante o horário permitido de check-in do hotel, e na data marcada.
Pós-condições	O status da(s) estadia(s) e do(s) cliente(s) envolvido(s) nela(s) é alterado para refletir no check-in realizado.
Fluxo principal	
Ações do Autor	Ações do sistema
1. Ator entra em contato com o cliente presencialmente no alojamento. Se cliente aparecer num horário inválido para check-in no estabelecimento, fluxo é	

redirecionado ao Fluxo alternativo 1	
2. Ator realiza o check-in do cliente na estadia	
	3. Servidor recebe a requisição de confirmação de check-in. Se o check-in for realizado numa data posterior ao combinado, redireciona ao Fluxo alternativo 2.
	4. Servidor confirma a realização do check-in e envia feedback positivo para a interface.
5. Ator entrega chaves do quarto para clientes e estadia é iniciada.	
Fluxo alternativo 1 – Tentativa de check-in em horário inválido	
Ações do Ator	Ações do sistema
	1. Aplicação do PMS bloqueia a realização de check-in em horário inválido.
2. Ator precisa comunicar aos clientes que o check-in não pode ser realizado, ou negociar a possibilidade de realizar o check-in mesmo assim. Se as regras do hotel permitirem o check-in, ação volta pro Fluxo principal, número 2. Se não, o fluxo fica a mercê das regras do estabelecimento.	
Fluxo alternativo 2 – Check-in em data posterior ao marcado.	
Ações do Ator	Ações do sistema
	1. Sistema identifica check-in em data posterior ao marcado, e realiza comportamento configurado pela empresa: cobrança extra, multa, bloqueio de check-in, etc.

tabela 16 – UC. 20 – Realizar check-out de estadia

Nome do caso de uso	UC. 20 → Realizar check-out de estadia
Ator principal	Funcionário de um hotel com permissão de realizar check-out de clientes, como um recepcionista.
Resumo	Ator principal realiza check-out em uma estadia previamente ativa por um cliente.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão de realizar check-out. Além disso, o cliente deve pedir para fazer check-out durante o horário permitido de check-out do hotel, e na data prevista ou antes.
Pós-condições	O status da(s) estadia(s) e do(s) cliente(s) envolvido(s) nela(s) é alterado para refletir no check-out realizado.

Fluxo principal

Ações do Autor	Ações do sistema
1. Ator entra em contato com o cliente presencialmente no alojamento. Se cliente aparecer num horário inválido para check-out no estabelecimento, fluxo é redirecionado ao Fluxo alternativo 1	
2. Ator realiza o check-out do cliente na estadia	
	3. Servidor recebe a requisição de confirmação de check-out. Se o check-out for realizado numa data anterior ao combinado, redireciona ao Fluxo alternativo 2. Se o check-out for realizado em uma data posterior ao combinado, redireciona ao Fluxo alternativo 3.
	4. Servidor confirma a realização do check-out e envia feedback positivo para a interface.
5. Ator recebe as chaves do quarto dos clientes e a estadia é encerrada.	

Fluxo alternativo 1 – Tentativa de check-out em horário inválido

Ações do Ator	Ações do sistema
	1. Aplicação do PMS bloqueia a realização de check-out em horário inválido.
2. Ator precisa comunicar aos clientes que o check-out não pode ser realizado, ou negociar a possibilidade de realizar o check-out mesmo assim. Se as regras do hotel permitirem o check-out, ação volta pro Fluxo principal, número 2. Se não, o fluxo depende das condições do hotel.	
Fluxo alternativo 2 – Check-out em data posterior ao marcado.	
Ações do Ator	Ações do sistema
	1. Sistema identifica check-out em data posterior ao marcado, e realiza comportamento configurado pela empresa: cobrança extra, multa, etc.
Fluxo alternativo 3 – Check-out em data anterior ao marcado.	
Ações do Ator	Ações do sistema
	1. Sistema identifica check-out em data anterior ao marcado, e realiza comportamento configurado pela empresa: restituição de valor, liberação, bloqueio, etc.

tabela 16 – UC. 20 – Realizar check-out de estadia

Nome do caso de uso	UC. 20 → Realizar check-out de estadia
Ator principal	Funcionário de um hotel com permissão de realizar check-out de clientes, como um recepcionista.
Resumo	Ator principal realiza check-out em uma estadia previamente ativa por um cliente.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão de realizar check-out. Além disso, o cliente deve pedir para fazer check-

	out durante o horário permitido de check-out do hotel, e na data prevista ou antes.
Pós-condições	O status da(s) estadia(s) e do(s) cliente(s) envolvido(s) nela(s) é alterado para refletir no check-out realizado.

Fluxo principal

Ações do Autor	Ações do sistema
1. Ator entra em contato com o cliente presencialmente no alojamento. Se cliente aparecer num horário inválido para check-out no estabelecimento, fluxo é redirecionado ao Fluxo alternativo 1	
2. Ator realiza o check-out do cliente na estadia	
	3. Servidor recebe a requisição de confirmação de check-out. Se o check-out for realizado numa data anterior ao combinado, redireciona ao Fluxo alternativo 2. Se o check-out for realizado em uma data posterior ao combinado, redireciona ao Fluxo alternativo 3.
	4. Servidor confirma a realização do check-out e envia feedback positivo para a interface.
5. Ator recebe as chaves do quarto dos clientes e a estadia é encerrada.	

Fluxo alternativo 1 – Tentativa de check-out em horário inválido

Ações do Ator	Ações do sistema
	1. Aplicação do PMS bloqueia a realização de check-out em horário inválido.
2. Ator precisa comunicar aos clientes que o check-out não pode ser realizado, ou negociar a possibilidade de realizar o check-out mesmo assim. Se as regras do hotel permitirem o check-out, ação volta pro Fluxo principal, número 2. Se não, o fluxo depende das condições do hotel.	

Fluxo alternativo 2 – Check-out em data posterior ao marcado.	
Ações do Ator	Ações do sistema
	1. Sistema identifica check-out em data posterior ao marcado, e realiza comportamento configurado pela empresa: cobrança extra, multa, etc.
Fluxo alternativo 3 – Check-out em data anterior ao marcado.	
Ações do Ator	Ações do sistema
	1. Sistema identifica check-out em data anterior ao marcado, e realiza comportamento configurado pela empresa: restituição de valor, liberação, bloqueio, etc.

tabela 17 – UC. 20 – Registrar consumo em estadia

Nome do caso de uso	UC. 20 → Realizar consumo em estadia	
Ator principal	Funcionário de um hotel com permissão de registrar consumo de clientes durante estadia, como um barman.	
Resumo	Ator principal registra o consumo de algum produto adicional por um cliente durante uma estadia ativa.	
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão de registrar consumo de um cliente. O registro deve ser apenas mediante a consumo real de produtos e o cliente deve estar consciente da cobrança.	
Pós-condições	O Produto consumido é registrado na conta do cliente.	
Fluxo principal		
Ações do Autor		Ações do sistema
1. Ator recebe contato de que o cliente deseja algum produto disponível nas		

dependencias do hotel.	
2. O consumo do produto pelo cliente é registrado pelo Ator, mas dependendo das regras do Hotel, o pagamento pode ser realizado na hora, em caso de serviço terceirizado, nesse caso, vai pro Fluxo alternativo 1.	
	3. Servidor recebe a requisição de confirmação de produto adquirido por cliente.
	4. Servidor confirma a operação de adicionar o produto consumido na conta do cliente e envia feedback positivo para a interface.
Fluxo alternativo 1 – Pagamento realizado na hora	
Ações do Ator	Ações do sistema
	1. Servidor recebe a requisição de confirmação de produto adquirido por cliente.
	2. Servidor confirma a operação de registrar o produto consumido pelo cliente e envia feedback positivo para a interface, mas não adiciona o produto como pendente para pagamento posterior.
2. Ator cobra o valor do produto do cliente na hora.	

tabela 18 – UC. 21 e UC. 23 – Registrar ou editar produto registrado

Nome do caso de uso	UC. 21 e UC. 23 → Registrar ou editar produto registrado
Ator principal	Funcionário de um hotel com permissão de registrar produto, como um gerente.
Resumo	Ator principal registra um novo produto disponível para

	aquisição pelos clientes ndurante suas estadias, ou edita os dados de um produto já cadastrado.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão de registrar e/ou editar produtos.
Pós-condições	O Produto é registrado nas dependências do serviço de hotelaria do Ator, ou tem seus dados editados. Os dados registrados do produto devem bater com os dados do produto fisicamente, como preço, disponibilidade, etc.
Fluxo principal	
Ações do Ator	Ações do sistema
1. Ator registra o produto na interface do PMS para o hotel que presta serviço ou edita os campos de um produto existente.	
2. Ator confirma o registro/edição.	
	3. Servidor recebe a requisição de registro ou edição do produto. No caso de falta de campos obrigatórios ou erro no preenchimento dos campos, o fluxo é redirecionado ao Fluxo alternativo 1.
	4. Servidor confirma a operação de adicionar o produto consumido na conta do cliente e envia feedback positivo para a interface.
Fluxo alternativo 1 – Erro de requisição	
Ações do Ator	Ações do sistema
	1. Servidor retorna mensagem de erro para o cliente, informando o caráter do erro.

tabela 19 – UC. 22 – Apagar produto

Nome do caso de uso	UC. 22 → Apagar produto
Ator principal	Funcionário de um hotel com permissão de apagar pro-

	duto, como um gerente.
Resumo	Ator principal apaga um produto previamente disponível para aquisição pelos clientes durante suas estadias.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão de apagar produtos.
Pós-condições	O Produto é apagado nas dependências do serviço de hotelaria do Ator.
Fluxo principal	
Ações do Ator	Ações do sistema
1. Ator seleciona o produto na interface do PMS ao qual deseja deletar do hotel que trabalha.	
2. Ator confirma a remoção do produto do escopo do hotel.	
	3. Servidor recebe a requisição de remoção.
	4. Servidor confirma a operação de remoção do produto e envia feedback positivo para a interface.

Tabela 20 – UC. 24 – Registrar pagamento

Nome do caso de uso	UC. 24 → Registrar pagamento
Ator principal	Funcionário de um hotel com permissão de registrar pagamento.
Resumo	Ator principal registra um pagamento realizado para o hotel.
Pré-condições	Ator principal deve estar conectado com suas credenci-

	as no sistema e possuir permissão de registrar pagamento. O registro de pagamento deve refletir um pagamento de fato realizado nas pendências da instituição.
Pós-condições	O pagamento registrado é somado as métricas financeiras do hotel.
Fluxo principal	
Ações do Autor	Ações do sistema
1. Ator recebe um pagamento relacionado ao hotel.	
2. Ator registra o pagamento na interface financeira do PMS.	
	3. Servidor recebe a requisição de registro de pagamento. Se houver falha no preenchimento de dados do registro ou campos obrigatórios faltando, fluxo redireciona ao Fluxo alternativo 1
	4. Servidor confirma a operação de registro de pagamento e envia feedback positivo para a interface.
Fluxo alternativo 1 – Erro de requisição	
Ações do Ator	Ações do sistema
	1. Servidor retorna mensagem de erro para o cliente, informando o caráter do erro.

Tabela 21 – UC. 25 – Gerar relatório financeiro

Nome do caso de uso	UC. 25 → Gerar relatório financeiro
Ator principal	Funcionário de um hotel com permissão de gerar relatório financeiro.
Resumo	Ator principal interage com ferramenta da interface do PMS que gera relatório financeiro do hotel ativo.

Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão de gerar relatório financeiro.
Pós-condições	O relatório é gerado como pdf.
Fluxo principal	
Ações do Autor	Ações do sistema
1. Ator interage com ferramenta de gerar relatório financeiro, selecionando pra receber um relatório completo ou de um recorte específico.	
	2. Sistema emite relatório em PDF.

tabela 22 – UC. 26 e UC. 27 – Criar ou editar temporada existente

Nome do caso de uso	UC. 26 e UC. 27 → Criar ou editar temporada existente.
Ator principal	Funcionário de um hotel com permissão de registrar nova temporada ou editar temporada existente, como um gerente.
Resumo	Ator principal registra uma nova temporada no hotel ou edita uma já existente.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão de registrar e/ou editar temporadas.
Pós-condições	A temporada nova ou editada é registrada ao sistema.
Fluxo principal	
Ações do Ator	Ações do sistema
1. Ator registra a temporada na interface	

do PMS para o hotel que presta serviço ou edita os campos de uma temporada existente.	
2. Ator confirma o registro/edição.	
	3. Servidor recebe a requisição de registro ou edição da temporada. No caso de falta de campos obrigatórios ou erro no preenchimento dos campos, o fluxo é redirecionado ao Fluxo alternativo 1.
	4. Servidor confirma a operação e envia feedback positivo para a interface.
Fluxo alternativo 1 – Erro de requisição	
Ações do Ator	Ações do sistema
	1. Servidor retorna mensagem de erro para o cliente, informando o caráter do erro.

tabela 23 – UC. 28 – Criar tarifa sob temporada

Nome do caso de uso	UC. 28 → Criar tarifa sob temporada
Ator principal	Funcionário de um hotel com permissão de registrar nova tarifa sob hotel mediante a temporada, como um gerente.
Resumo	Ator principal registra uma nova tarifa sob temporada no hotel.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão de registrar tarifas de temporada.
Pós-condições	A tarifa nova é registrada ao sistema.

Fluxo principal	
Ações do Ator	Ações do sistema
1. Ator registra a tarifa por quarto por temporada na interface do PMS para o hotel que presta serviço ou edita os campos de uma temporada existente.	
2. Ator confirma o registro/edição.	
	3. Servidor recebe a requisição de registro. No caso de falta de campos obrigatórios ou erro no preenchimento dos campos, o fluxo é redirecionado ao Fluxo alternativo 1.
	4. Servidor confirma a operação e envia feedback positivo para a interface.
Fluxo alternativo 1 – Erro de requisição	
Ações do Ator	Ações do sistema
	1. Servidor retorna mensagem de erro para o cliente, informando o caráter do erro.

tabela 24 – UC. 30 – Deletar temporada existente

Nome do caso de uso	UC. 30 → Deletar temporada existente
Ator principal	Funcionário de um hotel com permissão de deletar temporada existente.
Resumo	Ator principal deleta uma temporada no hotel.
Pré-condições	Ator principal deve estar conectado com suas credenciais no sistema e possuir permissão de deletar temporadas.

Pós-condições	A temporada selecionada é deletada.	
Fluxo principal		
Ações do Ator	Ações do sistema	
1. Ator seleciona a temporada que deseja deletar		
2. Ator confirma a remoção		
	3. Servidor recebe a requisição de remoção.	
	4. Servidor confirma a operação e envia feedback positivo para a interface.	

3.3 Arquitetura e código

3.3.1 Estrutura do projeto

O sistema foi organizado como um monorepo, isto é, um único repositório que concentra as aplicações e bibliotecas relacionadas ao produto. Essa decisão possibilita centralização do versionamento, padronização de ferramentas de desenvolvimento e manter os contratos entre cliente e servidor consistentes. O gerenciamento dos pacotes é realizado pelo PNPM, enquanto o Turborepo é responsável por gerenciar tarefas como desenvolvimento, compilação, testes e análise estática

Representação da estrutura do projeto.

```
/
├─ apps/
│   ├─ backend-service/
│   ├─ booking-engine-service/
│   └─ pms/
├─ packages/
│   └─ shared/
├─ notes/
├─ package.json
├─ pnpm-workspace.yaml
└─ turbo.json
```

Fontes: Autor

A separação entre diretórios `/apps` e `/packages` foi adotada para distinguir aplicações executáveis de código reutilizável. O papel do diretório `apps` é concentrar os serviços e interfaces que compõem o sistema, enquanto o da pasta `packages` é reunir elementos independentes de interface, como tipos TypeScript, contratos de autenticação, permissões, validações, internacionalização e funções financeiras, que são elementos utilizados por todos os serviços do sistema

A utilização do Turborepo também permite respeitar dependências entre os pacotes. Por exemplo, o pacote compartilhado pode ser compilado antes das aplicações que o consomem, enquanto tarefas de teste e verificação de tipos são executadas de forma padronizada em todo o repositório.

3.3.2 Servidor

O servidor está concentrado na aplicação `backend-service`, desenvolvida em Node.js e TypeScript, utilizando o framework Fastify. Sua responsabilidade é centralizar as regras de negócio, a autenticação, a autorização, o acesso ao banco de dados e a disponibilização de uma API HTTP para os clientes do sistema.

A aplicação é estruturada em camadas. As rotas recebem e validam as requisições HTTP; os repositórios encapsulam o acesso aos dados no Supabase; e os módulos comuns concentram funcionalidades transversais, como autenticação, tratamento de erros, geração de códigos de reserva e controle do hotel ativo. Essa organização reduz o acoplamento entre a API e a persistência dos dados, favorecendo a manutenção e os testes automatizados.

O backend disponibiliza recursos para gerenciamento de hotéis, usuários, papéis, permissões, quartos, hóspedes, produtos, temporadas, tarifas, reservas, hospedagens e transações financeiras. Há também operações específicas para o painel de reservas, incluindo check-in, check-out, cancelamento, registro de não comparecimento e lançamento de pagamentos.

A segurança é tratada no próprio serviço. As senhas são armazenadas por meio de hash Argon2id, e as sessões são representadas por tokens assinados com HMAC e prazo de validade de oito horas.

O sistema possui controle de tentativas de autenticação, bloqueando temporariamente a conta após dez falhas consecutivas. Além disso, cada operação é protegida por permissões e pelo contexto do hotel ativo, impedindo que usuários operem dados de hotéis aos quais não possuem acesso. A política de CORS também restringe as origens autorizadas a consumir a API.

3.3.3 Cliente

O cliente administrativo consiste no PMS (Property Management System), localizado em apps/pms. Ele foi desenvolvido com React, TypeScript e Next.js, utilizando o App Router para a organização de páginas, layouts e ações de servidor. A estilização é realizada com Tailwind CSS.

O PMS constitui a interface utilizada pelos colaboradores para executar atividades operacionais e administrativas. Entre suas funcionalidades estão a autenticação de usuários, seleção do hotel ativo, administração de hotéis, usuários, permissões e papéis, cadastro de quartos, hóspedes e produtos, definição de temporadas

e tarifas, gerenciamento de reservas, acompanhamento de hospedagens e controle financeiro.

A interface não acessa o banco de dados diretamente. As requisições são encaminhadas ao backend-service, que valida a sessão, as permissões e o escopo do hotel antes de realizar qualquer operação. O token de sessão é armazenado pelo PMS em cookie HttpOnly, reduzindo sua exposição ao código executado no navegador. A navegação e os módulos exibidos também são condicionados às permissões do usuário autenticado.

Essa separação mantém o PMS responsável pela apresentação dos dados e pela interação com o usuário, enquanto as regras de negócio permanecem concentradas no backend.

3.3.4 Site institucional

O site institucional está previsto para uma etapa futura do projeto e ainda não possui implementação no repositório. Sua finalidade será apresentar a solução de hotelaria ao público externo, disponibilizando informações institucionais, funcionalidades do sistema, formas de contato e, quando aplicável, meios de direcionamento para reservas ou demonstrações da plataforma.

A proposta é desenvolvê-lo com React, preferencialmente por meio do Next.js, que é um framework baseado em React já adotado no PMS. Essa escolha permite reutilizar conhecimentos, padrões de desenvolvimento e parte da infraestrutura do projeto. Além disso, o Next.js oferece recursos adequados a um site público, como renderização no servidor, geração estática de páginas e melhores condições para otimização em mecanismos de busca.

O site institucional não deverá concentrar regras de negócio nem realizar acesso direto ao banco de dados. Quando houver necessidade de informações dinâmicas, como disponibilidade, formulários ou integrações com reservas, a comunicação deverá ocorrer por meio das APIs disponibilizadas pelo backend.

3.3.5 Banco de dados

A persistência dos dados é realizada no Supabase, utilizando sua base PostgreSQL gerenciada. O Supabase é empregado como infraestrutura de banco de dados e como interface de acesso programático a recursos do PostgreSQL, enquanto as decisões de negócio e de autorização permanecem sob responsabilidade do backend-service.

O modelo de dados contempla as principais entidades do domínio hoteleiro: hotéis, usuários, papéis, permissões, quartos, hóspedes, reservas, hospedagens, produtos, temporadas, tarifas sazonais e transações financeiras. As relações são implementadas com chaves estrangeiras e tabelas associativas, especialmente para a vinculação entre usuários, papéis e permissões. Esse modelo permite que um usuário possua diferentes papéis e permissões em contextos distintos de hotel.

O sistema adota uma estrutura compatível com múltiplos hotéis. As entidades operacionais possuem referência ao hotel correspondente, permitindo isolar os dados de cada estabelecimento. O backend utiliza o contexto do hotel ativo para filtrar e validar as operações realizadas pelo usuário.

A integridade dos dados é reforçada no banco por meio de restrições, chaves estrangeiras, índices, triggers e funções SQL. Entre as regras implementadas estão a validação de valores monetários não negativos, a capacidade máxima dos quartos, a consistência entre datas de entrada e saída, a prevenção de conflitos de reservas para um mesmo quarto e a atualização automática de campos de auditoria, como `updated_at`.

O acesso ao Supabase ocorre exclusivamente pelo backend, por meio de uma credencial administrativa mantida em variáveis de ambiente. Portanto, o navegador não recebe credenciais de banco de dados e não realiza consultas diretas. A autenticação da aplicação não utiliza o Supabase Auth: os usuários são mantidos em tabela própria, as senhas são protegidas por hash Argon2id e as sessões são controladas pelo serviço de backend.

4. Conclusão

O desenvolvimento deste projeto possibilitou a aplicação prática de aprendi-

zados adquiridos ao longo da formação de Análise e Desenvolvimento de Sistemas, envolvendo etapas de planejamento, modelagem, implementação, testes e documentação de uma solução cujo objetivo é se aproximar de padrões de mercado.

Apesar do sistema e a documentação ainda necessitarem de muitos aprimoramentos e ferramentas ainda não implementadas, esta etapa conclui a viabilidade do projeto e demonstra competência para criação de uma solução próxima ao mundo real.